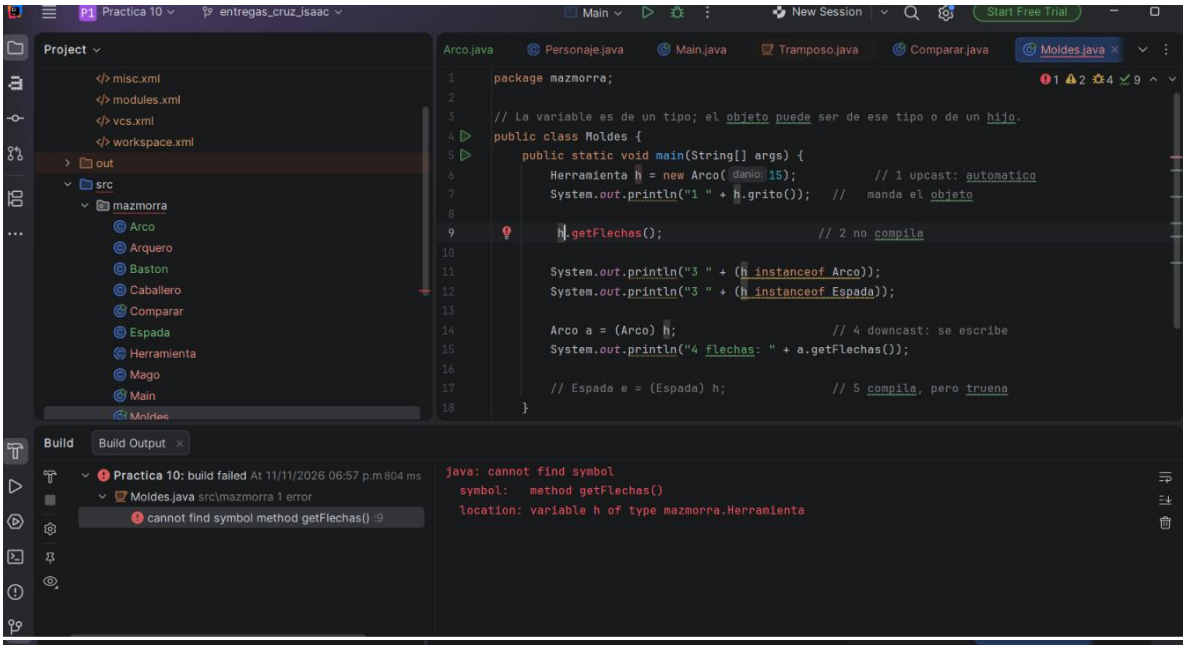
BITÁCORA - PRÁCTICA 10: LA MAZMORRA (Polimorfismo, primera parte)

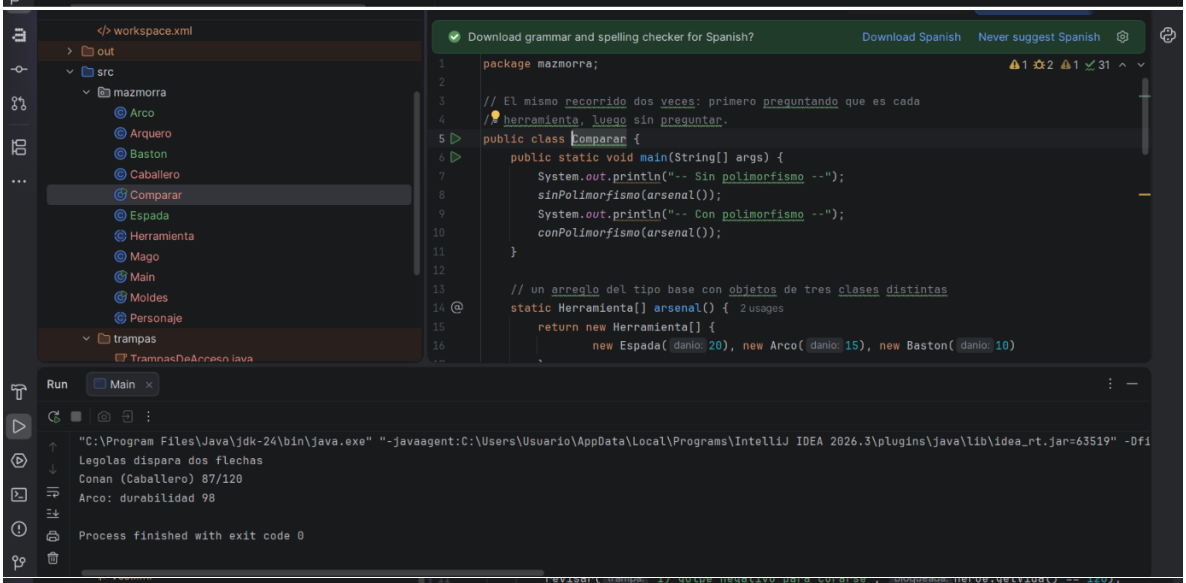
- **Integrantes:**
  - Isaac Cruz Chávez (Número de cuenta: 424020705)

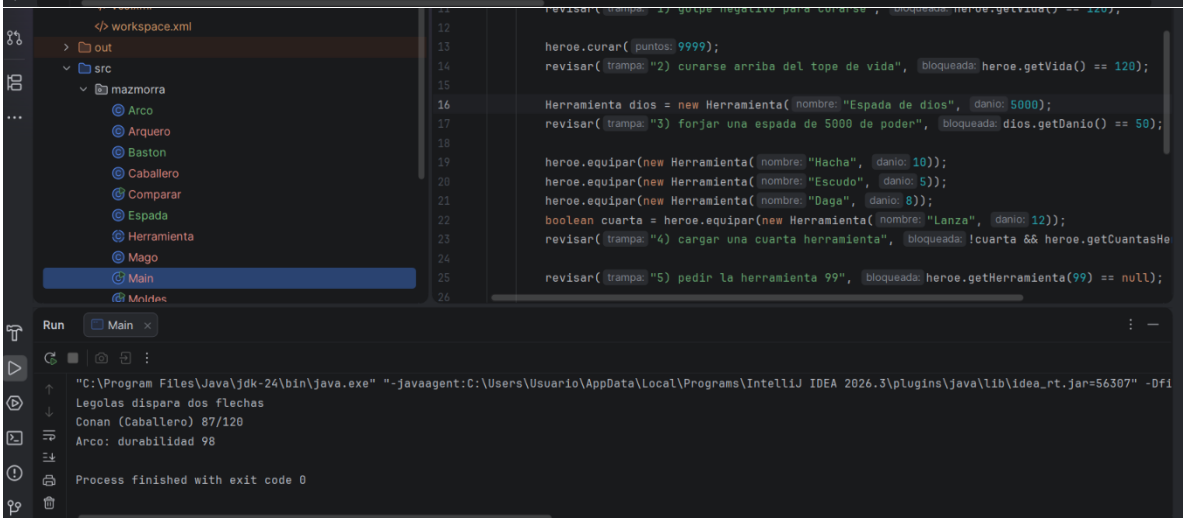
- **Grupo:** 6

- **Materia:** Modelos de Programación Orientada a Objetos (2027-1)

1. Capturas de Evidencia Obligatorias


Practica 10
entregas_cruz_isaac
Main
New Session
Start Free Trial
Project
misc.xml
modules.xml
vcs.xml
workspace.xml
out
src
mazmorra
Arco
Arquero
Baston
Caballero
Comparar
Espada
Herramienta
Mago
Main
Moldes
Arco.java
Personaje.java
Main.java
Tramposo.java
Comparar.java
Moldes.java
package mazmorra;
// La variable es de un tipo; el objeto puede ser de ese tipo o de un hijo.
public class Moldes {
public static void main(String[] args) {
Herramienta h = new Arco(danio: 15); // 1 upcast: automatico
System.out.println("1 " + h.grito()); // manda el objeto
h.getFlechas(); // 2 no compila
System.out.println("3 " + (h instanceof Arco));
System.out.println("3 " + (h instanceof Espada));
Arco a = (Arco) h; // 4 downcast: se escribe
System.out.println("4 flechas: " + a.getFlechas());
// Espada e = (Espada) h; // 5 compila, pero truena
}
Build
Build Output
Practica 10: build failed At 11/11/2026 06:57 p.m 804 ms
Moldes.java src\mazmorra 1 error
cannot find symbol method getFlechas() :9
java: cannot find symbol
symbol: method getFlechas()
location: variable h of type mazmorra.Herramienta


Download grammar and spelling checker for Spanish?
Download Spanish
Never suggest Spanish
package mazmorra;
// El mismo recorrido dos veces: primero preguntando que es cada
// herramienta, luego sin preguntar.
public class Comparar {
public static void main(String[] args) {
System.out.println("-- Sin polimorfismo --");
sinPolimorfismo(arsenal());
System.out.println("-- Con polimorfismo --");
conPolimorfismo(arsenal());
}
// un arreglo del tipo base con objetos de tres clases distintas
static Herramienta[] arsenal() { 2 usages
return new Herramienta[] {
new Espada(danio: 20), new Arco(danio: 15), new Baston(danio: 10)
Run
Main
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Users\Usuario\AppData\Local\Programs\IntelliJ IDEA 2026.3\plugins\java\lib\idea_rt.jar=63519" -Dfi
Legolas dispara dos flechas
Conan (Caballero) 87/120
Arco: durabilidad 98
Process finished with exit code 0


heroe.curar(puntos: 9999);
revisar(trampa: "2) curarse arriba del tope de vida", bloqueada: heroe.getVida() == 120);
Herramienta dios = new Herramienta(nombre: "Espada de dios", danio: 5000);
revisar(trampa: "3) forjar una espada de 5000 de poder", bloqueada: dios.getDanio() == 50);
heroe.equipar(new Herramienta(nombre: "Hacha", danio: 10));
heroe.equipar(new Herramienta(nombre: "Escudo", danio: 5));
heroe.equipar(new Herramienta(nombre: "Daga", danio: 8));
boolean cuarta = heroe.equipar(new Herramienta(nombre: "Lanza", danio: 12));
revisar(trampa: "4) cargar una cuarta herramienta", bloqueada: !cuarta && heroe.getCuantasHe
revisar(trampa: "5) pedir la herramienta 99", bloqueada: heroe.getHerramienta(99) == null);
Run
Main
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Users\Usuario\AppData\Local\Programs\IntelliJ IDEA 2026.3\plugins\java\lib\idea_rt.jar=56307" -Dfi
Legolas dispara dos flechas
Conan (Caballero) 87/120
Arco: durabilidad 98
Process finished with exit code 0

- **A. Salida de** Main.java**:** Muestra la ejecución donde Legolas ataca a Conan (Legolas dispara dos flechas, vida de Conan en 87/120 y durabilidad del Arco en 98).

- **B. Salida de** Comparar.java**:** Muestra ambas secciones (-- Sin polimorfismo -- y -- Con polimorfismo --) imprimiendo exactamente las mismas 3 líneas para Espada, Arco y Bastón.

- **C. Error de la línea 2 en** Moldes.java**:** Captura del error de compilación cannot find symbol: method getFlechas().

- **D. Error de la línea 5 en** Moldes.java**:** Captura de la excepción al correr java.lang.ClassCastException.

- **E. Salida limpia de** Moldes.java**:** Ejecución normal mostrando las impresiones (1 dispara..., 3 true, 3 false, 4 flechas: 2).

2. Respuestas Teóricas

1. Una orden, muchas respuestas

- **Línea que llama a** golpear()**:** En la clase Personaje, la llamada a la orden se ejecuta dentro del método atacar(Personaje objetivo) en la línea:

Java

int base = herramienta.golpear();

```[cite: 11]

- **¿Quién decidió qué?**
```

    - Las **dos flechas** las decidió la clase Arco al implementar su propio método golpear(), el cual devuelve la suma de dos usos (return usar() + usar();).

    - El **+3** de daño extra lo decidió la clase Arquero al sobrescribir el método calcularDanio(int base).

- **¿Qué tipo decidió?** En ambos casos lo decidió el **tipo del objeto en memoria** (a través del enlace dinámico en tiempo de ejecución), no el tipo de la variable que los referencia (Herramienta y Personaje).

2. Sin y con polimorfismo

- **Si mañana llega un** Hacha**:**

    - **En la mitad *Sin polimorfismo*:** Habría que modificar el código de Comparar.java agregando una nueva condición else if (h instanceof Hacha). Si alguien lo olvida y no actualiza esa parte, el programa cae en la rama por defecto e imprime "no se que es esto" con daño 0.

    - **En la mitad *Con polimorfismo*:** No hay que cambiar absolutamente nada en Comparar.java. Basta con crear la clase Hacha heredando de Herramienta e implementando sus métodos abstractos. Al recorrer el arreglo, el ciclo for ejecutará automáticamente el grito() y golpear() propios del objeto Hacha.

3. Los moldes (Explicación de errores)

- **Línea 2 (**h.getFlechas();**):**

    - **Error observado:** cannot find symbol: method getFlechas().

    - **Explicación:** El problema lo encontró el **compilador (al hacer** javac**)**. El compilador solo revisa el **tipo de la variable** (Herramienta), y como la clase abstracta Herramienta no tiene declarado el método getFlechas(), no permite compilar aunque en la memoria el objeto real sea un Arco.

- **Línea 5 (**Espada e = (Espada) h;**):**

    - **Error observado:** java.lang.ClassCastException: class mazmorra.Arco cannot be cast to class mazmorra.Espada.

    - **Explicación:** El problema lo encontró **Java al correr el programa (la JVM)**. El compilador aprueba la línea porque sintácticamente es válido hacer un *downcast* de una variable de tipo base a un tipo hijo. Sin embargo, al ejecutar la instrucción, la máquina virtual revisa el **tipo real del objeto** en la memoria RAM y, al ver que es un Arco y no una Espada, detiene la ejecución lanzando la excepción.